Ghulam Ishaq Khan Institute (GIKI)

Quiz # 2

Subject: Object Oriented Programming	**Course Code:** CS - 112 – Spring - 2025
Class: BS CY **Batch:** Fall – 2024	**Quiz Date:** 18 March
Course Instructor: Muhammad Ahmad Nawaz	**Total Marks:** 12
Course TA:	**Total Time:** 15 Minutes
Name:	Registration Number:

Note (Read notes & instructions first)

- First, read the instructions and statements of each question carefully.
  - Attempt all questions
  - Don't overwrite.
  - **CHEATING/COPY CASE will be graded as STRAIGHT ZERO MARKS.**

Fix Errors and write outputs

Program 1

```
#include <iostream>
class Shape { public:
   void draw() { cout << "Shape\n"; }
};
class Circle : public Shape {
public:
   void draw() { cout << "Circle\n"; }
};
int main() {
   Shape* s = new Circle();
   s->draw();
   delete s;
   return 0; }
```

Program 2

```
#include <iostream>
using namespace std;

class A { public: A() { cout << "A "; } };
class B : public A { public: B() { cout << "B "; } };
class C : public B { public: C() { cout << "C "; } };

int main() {
   C obj;
   return 0;
}
```

Program 3

```
#include <iostream>
using namespace std;

class Base {
public:
   virtual void show() { cout << "Base\n"; }
};

class Derived : public Base {
public:
   void show() override { cout << "Derived\n"; }
};

int main() {
   Base* b = new Derived();
   b->show();
   delete b;
   return 0;
}
```

Program 4

```
#include <iostream>
using namespace std;
class Base1 { public: void func() { cout << "Base1\n"; } };
class Base2 { public: void func() { cout << "Base2\n"; } };
class Derived : public Base1, public Base2 {};
int main() {
   Derived d;
   d.func();
   return 0; }
```

Program 1

The error in the code is related to the use of `cout` without the `std::` namespace qualifier. In C++, `cout` is part of the `std` namespace, so you need to either:

1. Use `std::cout` explicitly, or
2. Add `using namespace std;` at the beginning of the file to bring the `std` namespace into scope.

Here's the corrected code:

```
#include <iostream>

class Shape {
public:
   void draw() { std::cout << "Shape\n"; }  // Use std::cout
};

class Circle : public Shape {
public:
   void draw() { std::cout << "Circle\n"; }  // Use std::cout
};

int main() {
   Shape* s = new Circle();
   s->draw();  // This will call Shape::draw(), not Circle::draw()
   delete s;
   return 0;
}
```

The **output** of the original code (without the `virtual` keyword) will be: Shape

If you fix the code by adding the `virtual` keyword:

Here's the corrected code:

```
#include <iostream>

class Shape {
public:
   virtual void draw() { std::cout << "Shape\n"; }  // Declare as virtual
};

class Circle : public Shape {
public:
   void draw() override { std::cout << "Circle\n"; }  // Override the virtual function
};

int main() {
   Shape* s = new Circle();
   s->draw();  // This will now call Circle::draw()
   delete s;
   return 0;
}
```

The **output** will now be: Circle

Program 2

The code is correct, and there are **no errors**. The output will be `A B C`.

Program 3

The code is correct, and there are **no errors**. The output will be `Derived`.

Program 4

The **error** in the code is due to **ambiguity** in calling the `func()` method. The `Derived` class inherits from both `Base1` and `Base2`, and both base classes have a function named `func()`. When you try to call `d.func()`, the compiler cannot determine which `func()` to call (`Base1::func()` or `Base2::func()`).

Error Message:

The compiler will throw an error similar to:

Copy

```
error: request for member 'func' is ambiguous
```

Fix:

To resolve the ambiguity, you need to explicitly specify which `func()` you want to call. For example:

```
#include <iostream>
using namespace std;

class Base1 { public: void func() { cout << "Base1\n"; } };
class Base2 { public: void func() { cout << "Base2\n"; } };
class Derived : public Base1, public Base2 {};

int main() {
   Derived d;
   d.Base1::func();  // Explicitly call Base1's func()
   d.Base2::func();  // Explicitly call Base2's func()
   return 0;
}
```

Output:

```
Base1
Base2
```

The error is due to **ambiguity** in calling `func()`. You need to explicitly specify which base class's `func()` to call.